

```

from copy import deepcopy

MOVES = {
    "Up": (-1, 0),
    "Down": (1, 0),
    "Left": (0, -1),
    "Right": (0, 1)
}

def input_state(name):
    print(f"\nEnter {name} state (use 0 for blank):")
    state = []

    for i in range(3):
        row = list(map(int, input(f"Row {i + 1}: ").split()))

        while len(row) != 3:
            print("Enter exactly 3 numbers.")
            row = list(map(int, input(f"Row {i + 1}: ").split()))

        state.append(row)

    return state

def print_state(state):
    for row in state:
        print(" ".join(map(str, row)))
    print()

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Misplaced Tiles heuristic
def misplaced_tiles(state, goal):
    count = 0

    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1

    return count

def get_neighbors(state):
    x, y = find_blank(state)

    for move, (dx, dy) in MOVES.items():
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = deepcopy(state)

            new_state[x][y], new_state[nx][ny] = \
                new_state[nx][ny], new_state[x][y]

            yield new_state, move

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def best_first_search(initial, goal):
    # (heuristic, state, path, moves)
    queue = [

```

```
(misplaced_tiles(initial, goal), initial, [initial], [])
]
```

```
visited = set()

while queue:

    # Select state with smallest heuristic value
    queue.sort(key=lambda x: x[0])
    h, state, path, moves = queue.pop(0)

    state_tuple = state_to_tuple(state)

    if state_tuple in visited:
        continue

    visited.add(state_tuple)

    # Goal reached
    if state == goal:
        return path, moves

    # Generate next states
    for new_state, move in get_neighbors(state):

        new_tuple = state_to_tuple(new_state)

        if new_tuple not in visited:
            h_value = misplaced_tiles(new_state, goal)

            queue.append(
                (
                    h_value,
                    new_state,
                    path + [new_state],
                    moves + [move]
                )
            )

    return None, None
```

```
# ----- MAIN -----
```

```
initial = input_state("initial")
goal = input_state("goal")

print("\n===== INITIAL STATE =====")
print_state(initial)

print("===== GOAL =====")
print_state(goal)

print("===== MISPLACED TILES SEARCH =====\n")

path, moves = best_first_search(initial, goal)

if path is None:
    print("No solution found.")

else:
    print("===== SOLUTION =====\n")

    for i, state in enumerate(path):

        h = misplaced_tiles(state, goal)

        if i == 0:
            print("Initial State:")
        else:
            print(f"Move {i}: {moves[i - 1]}")

        print(f"Misplaced Tiles = {h}")
        print_state(state)

    print("Goal reached!")
    print("Total moves:", len(moves))
```

Enter initial state (use 0 for blank):

Row 1: 1 2 3

Row 2: 4 0 6

Row 3: 7 5 8

Enter goal state (use 0 for blank):

Row 1: 1 2 3

Row 2: 4 5 6

Row 3: 7 8 0

===== INITIAL STATE =====

1 2 3

4 0 6

7 5 8

===== GOAL =====

1 2 3

4 5 6

7 8 0

===== MISPLACED TILES SEARCH =====

===== SOLUTION =====

Initial State:

Misplaced Tiles = 2

1 2 3

4 0 6

7 5 8

Move 1: Down

Misplaced Tiles = 1

1 2 3

4 5 6

7 0 8

Move 2: Right

Misplaced Tiles = 0

1 2 3

4 5 6

7 8 0

Goal reached!

Total moves: 2